

ONE'S AND TWO'S COMPLEMENT

CS 6306 – Fundamentals Crash Course

Cyber Security Analytics and Malware Analysis for Practitioners

Course Description

A supervisory view of security vulnerabilities in information systems, applications and safe computing. Additionally, best practices to promote security standards and applications, management and analysis of security. Applications of data science techniques, preparation for adversarial attacks.

Course information

Course Title: Cyber Security Analytics and Malware Analysis for Practitioners
Section: 501
Course Term: Fall 2026

1. Binary From Absolute Zero

1.1 What is a bit?

A bit is one binary digit. A bit can only be 0 or 1. That is the entire alphabet of binary numbers.

- Decimal uses ten digits: 0 through 9.
- Binary uses two digits: 0 and 1.
- A group of 8 bits is called a byte.

Example: The bit string 0010 0101 contains 8 bits, so it is one byte. The space in the middle is only for readability; it does not change the value.

1.2 Binary place values

In decimal, place values are powers of 10: ones, tens, hundreds, and so on. In binary, place values are powers of 2. Starting from the rightmost bit, the values are 1, 2, 4, 8, 16, 32, 64, 128, and so on.

128	64	32	16	8	4	2	1
b7	b6	b5	b4	b3	b2	b1	b0

The top row is the value contributed by each position. A 1 means 'include this value.' A 0 means 'do not include it.'

Example A: Read 0010 0101 as an ordinary positive binary number

Step 1: Write the place values under the bits.

Bits:	0	0	1	0	0	1	0	1
Place value:	128	64	32	16	8	4	2	1
Included:	-	-	32	-	-	4	-	1

Step 2: Add the place values whose bits are 1: $32 + 4 + 1 = 37$.

Answer: 0010 0101 = 37 when interpreted as an ordinary unsigned binary number.

1.3 MSB and LSB

You will see these abbreviations in slides and quizzes:

- **MSB** = most significant bit = the leftmost bit. It has the largest place value.
- **LSB** = least significant bit = the rightmost bit. It has the smallest place value, 1.

When the lecture says something is happening on the 'LSB side,' it means the right-hand end of the bit string.

1.4 Unsigned vs. signed numbers

Unsigned means every bit is used only for magnitude, so the number cannot be negative. Signed means the bit pattern is being used to represent positive and negative values.

Important: For signed one's-complement and two's-complement numbers, the leftmost bit is called the sign bit. A sign bit of 0 means the value is nonnegative; a sign bit of 1 means the value is negative. However, you cannot simply erase the sign bit and read the rest. Negative values must be decoded using the complement rule.

1.5 Why bit width matters

The bit width is the number of bits you are allowed to keep. The slides mostly use 8-bit values. If a question says '8-bit,' you must keep exactly 8 bits, including leading zeros.

- 5 in ordinary binary is 101.
- 5 written as an 8-bit value is 0000 0101.
- The leading zeros are not optional when the width matters, because complement operations flip every bit in the fixed-width pattern.

2. Converting Decimal to Binary

The easiest beginner method for the numbers in these slides is the place-value method: choose the largest power of 2 that fits, subtract it, and continue.

Example B: Convert decimal 34 to 8-bit binary

Step 1: The 8-bit place values are 128, 64, 32, 16, 8, 4, 2, 1.

Step 2: 34 contains 32, leaving 2. So the 32 position gets a 1.

Step 3: 34 does not need 16, 8, or 4, so those positions are 0.

Step 4: The remaining 2 means the 2 position gets a 1. The 1 position is 0.

Place value:	128	64	32	16	8	4	2	1
Bit:	0	0	1	0	0	0	1	0
Result:	0010 0010							

Answer: 34 = 0010 0010 in 8-bit binary. This exact value is used in the lecture to build -34 in both complement systems.

Example C: Convert decimal 13 to 8-bit binary

Think: 13 = 8 + 4 + 1.

Place value:	128	64	32	16	8	4	2	1
Bit:	0	0	0	0	1	1	0	1
Result:	0000 1101							

Answer: 13 = 0000 1101. Later, we will use this to verify that a negative result really represents -13.

3. Binary Addition From Zero

Binary addition works like decimal column addition, but each column can contain only 0 or 1. When a column reaches the binary value 10, you write 0 and carry 1 to the next column on the left.

3.1 The four rules to memorize

Addition	Binary result	What you do
0 + 0	0	Nothing to carry.
0 + 1 or 1 + 0	1	Nothing to carry.
1 + 1	10	Write 0 in this column and carry 1 left.
1 + 1 + carried 1	11	Write 1 in this column and carry 1 left.

Example D: A carry-heavy warm-up: 7 + 5

This example is not from the slides; it is included because the lecture examples assume you already know how carries work.

```
0000 0111 (7)
+ 0000 0101 (5)
-----
0000 1100 (12)
```

Rightmost column: $1 + 1 =$ binary 10. Write 0; carry 1.

Next column: $1 + 0 +$ carried 1 = binary 10. Write 0; carry 1.

Next column: $1 + 1 +$ carried 1 = binary 11. Write 1; carry 1.

Next column: $0 + 0 +$ carried 1 = 1. Write 1. No more carry.

Check: 0000 1100 means $8 + 4 = 12$, so the binary addition is correct.

Do not mix up two different ideas: A carry is a normal part of column addition. Overflow is a separate idea meaning the true answer does not fit in the allowed signed range. We will handle overflow later.

4. One's Complement

The supplied slides introduce one's complement as a way to represent negative numbers and as a use of the bitwise NOT operation, written with the symbol $\sim$. The defining action is extremely simple: flip every bit.

4.1 The one's-complement rule

Rule: To form the one's complement of a fixed-width bit pattern, change every 0 to 1 and every 1 to 0.

0 becomes 1
1 becomes 0

If you are representing a negative decimal number in one's complement, first write the positive magnitude in the required number of bits, then flip every bit.

Example 1: Flip 0010 0101 [Lecture p. 3]

Question meaning: The lecture asks for the one's complement of the 8-bit pattern 0010 0101. You are not doing arithmetic yet; you are only flipping bits.

Original: 0 0 1 0 0 1 0 1
Flip each bit: 1 1 0 1 1 0 1 0
One's comp.: 1101 1010

Bit-by-bit check: The first 0 became 1, the next 0 became 1, the 1 became 0, and the same flip rule continues through all 8 positions.

Answer: $\sim(0010\ 0101) = 1101\ 1010$.

4.2 Why it is called 'one's complement'

The lecture explains the name by adding a number to its flipped version. At every bit position, one value has a 0 where the other has a 1. Therefore each column contributes 1, producing a result made entirely of 1s.

```
0010 0101
+ 1101 1010
-----
1111 1111
```

Meaning: The original and its one's complement fill each bit position with one 0 and one 1, so the sum is all 1s.

4.3 Representing a negative number in one's complement

For an n-bit one's-complement number:

1. Write the positive magnitude in exactly n bits.
2. Flip every bit.
3. Stop. Do not add 1. Adding 1 would change the method into two's complement.

Example 2: Represent -34 in 8-bit one's complement [Lecture p. 5]

Step 1: Ignore the negative sign temporarily and convert +34 to 8-bit binary.

```
34 = 32 + 2
+34 = 0010 0010
```

Step 2: Because the target is -34 in one's complement, flip all 8 bits.

```
+34: 0 0 1 0 0 0 1 0
flip: 1 1 0 1 1 1 0 1
-34: 1101 1101
```

Verification: Flip 1101 1101 again. You get 0010 0010, which is 34. Therefore the stored negative value represents -34.

Answer: -34 = 1101 1101 in 8-bit one's complement.

Common mistake: Writing 1101 1110 would be the two's-complement version of -34, not the one's-complement version.

4.4 How to decode a one's-complement bit pattern

To turn a one's-complement bit pattern back into decimal:

1. Look at the leftmost bit (the sign bit).
2. If it is 0, read the bit pattern as an ordinary positive binary number.
3. If it is 1, the value is negative. Flip every bit, convert the flipped pattern to decimal, then place a minus sign in front.

Special case: All zeros is +0. All ones is -0 in one's complement. This duplicated zero is one reason two's complement is preferred.

Example E: Decode 1111 1100 as an 8-bit one's-complement value

Step 1: The sign bit is 1, so the number is negative.

Step 2: Flip every bit: 1111 1100 -> 0000 0011.

Step 3: 0000 0011 is 3.

Answer: 1111 1100 represents -3 in 8-bit one's complement. This is the result that appears in the lecture's -8 + 5 example.

5. One's-Complement Addition: Every Lecture Example

The lecture uses ordinary binary column addition, with one special rule for one's complement: if a carry comes out past the leftmost bit, wrap that carry around and add it to the rightmost end. This is often called an end-around carry.

One's-complement carry rule: If there is a carry out beyond the fixed bit width, keep the lower n bits and add the carry value 1 to that n-bit result. In the lecture's wording, the 'discard carry' wraps around to the other end.

Example 3: Add +8 and +5 [Lecture p. 6]

Step 1: Both values are positive, so write normal 8-bit binary. No complement conversion is needed.

```
+8 = 0000 1000
+5 = 0000 0101
```

Step 2: Add exactly as ordinary unsigned binary numbers.

```
0000 1000 (8)
+ 0000 0101 (5)
-----
0000 1101 (13)
```

Step 3: The result begins with 0, so it is positive. $0000\ 1101 = 8 + 4 + 1 = 13$.

Carry check: No carry leaves the left side, so there is no end-around carry to add.

Answer: $+8 + +5 = +13$, stored as 0000 1101.

Example 4: Add -8 and +5 [Lecture p. 7]

Step 1: Convert -8 to one's complement. Start with $+8 = 0000\ 1000$ and flip every bit.

```
+8: 0000 1000
-8: 1111 0111 (one's complement)
```

Step 2: Write +5 normally as 0000 0101.

Step 3: Add the two 8-bit patterns.

```
1111 0111 (-8)
+ 0000 0101 (+5)
-----
1111 1100
```

Step 4: There is no carry beyond the leftmost bit, so no wraparound is needed.

Step 5: Decode 1111 1100 as one's complement. The sign bit is 1, so flip it: $0000\ 0011 = 3$. Attach the negative sign.

Answer: $-8 + 5 = -3$, stored as 1111 1100 in 8-bit one's complement.

What the slide means by LSB side: 1111 1100 ends in 00 on the rightmost, least-significant side. The key fact is not the shape itself; the reliable method is to flip the bits and decode the magnitude.

Example 5: Add -8 and -5 with end-around carry [Lecture pp. 8-9]

Step 1: Build -8: +8 is 0000 1000; flipping gives 1111 0111.

Step 2: Build -5: +5 is 0000 0101; flipping gives 1111 1010.

Step 3: Add them as ordinary binary values. This produces a ninth carry bit on the far left.

```
1111 0111 (-8)
+ 1111 1010 (-5)
-----
1 1111 0001 <- 9-bit raw sum
```

Step 4: Because this is one's-complement addition, do NOT simply throw away the leftmost carry. Keep the lower 8 bits 1111 0001, then wrap the carry 1 around and add it at the right.

```
1111 0001
+ 0000 0001 <- end-around carry
-----
1111 0010
```

Step 5: Decode 1111 0010. It starts with 1, so it is negative. Flip it: 0000 1101. That is 13.

Answer: 1111 0010 represents -13, so $-8 + -5 = -13$.

Critical contrast: In one's complement, the carry wraps around and gets added back. In two's complement, a carry past the left edge is simply discarded. This difference is one of the easiest exam points to confuse.

5.1 The duplicated-zero problem

The lecture identifies the main awkward feature of one's complement: zero has two bit patterns.

```
0000 0000 = +0
1111 1111 = -0
```

Mathematically, +0 and -0 are the same value, so keeping two encodings for zero wastes one bit pattern. The lecture then asks what happens if we take the one's-complement result and add 1. That leads directly to two's complement.

5.2 Range of one's-complement values

With n bits, one's complement has one sign bit and two encodings for zero. Its range is:

Formula: $-(2^{(n-1)} - 1)$ through $+(2^{(n-1)} - 1)$.

- For 4 bits: -7 through +7, plus the extra -0 encoding.
- For 8 bits: -127 through +127, plus the extra -0 encoding. This matches the lecture.

6. Two's Complement

Two's complement starts from the same bit-flipping idea but adds one more step. The lecture describes it as: take the one's-complement form and add 1.

6.1 The two's-complement rule

Rule: To represent $-x$ in n -bit two's complement: (1) write $+x$ in exactly n bits, (2) flip every bit, and (3) add 1. Keep only n bits.

Positive x -> fixed-width binary
Flip bits -> one's-complement form
+ 1 -> two's-complement negative form

6.2 Why adding 1 fixes zero

The lecture applies the rule to zero. This is the key reason two's complement has only one zero.

Example 6: Negate zero using two's complement [Lecture pp. 13-15]

Step 1: Start with 8-bit zero: 0000 0000.

Step 2: Flip every bit to get its one's complement: 1111 1111.

Step 3: Add 1.

```
1111 1111
+ 0000 0001
-----
1 0000 0000
```

Step 4: The allowed width is only 8 bits. Discard the ninth carry bit, leaving 0000 0000.

Result: The two's complement of zero is still zero. There is no separate negative-zero representation.

6.3 So what is 1111 1111 in two's complement?

The lecture answers: -1. We can prove it using the normal decoding method.

Step 1: The sign bit is 1, so the value is negative.

Step 2: Flip 1111 1111 -> 0000 0000.

Step 3: Add 1 -> 0000 0001.

Step 4: The magnitude is 1, so the original value is -1.

Answer: 1111 1111 = -1 in 8-bit two's complement, but it means -0 in 8-bit one's complement. Same bits, different representation, different meaning.

6.4 Represent -34 in two's complement

Example 7: Represent -34 in 8-bit two's complement [Lecture p. 17]

Step 1: Write +34 in 8 bits. As shown earlier, $34 = 32 + 2 = 0010\ 0010$.

Step 2: Flip every bit: 0010 0010 -> 1101 1101.

Step 3: Add 1 to the flipped result.

```
1101 1101
+ 0000 0001
-----
1101 1110
```

Answer: -34 = 1101 1110 in 8-bit two's complement.

Compare: One's complement -34 was 1101 1101. Two's complement is exactly one larger in the bit pattern because we add 1 after flipping.

6.5 How to decode a two's-complement bit pattern

To convert an n-bit two's-complement pattern back to decimal:

1. Look at the sign bit.
2. If the sign bit is 0, read it as ordinary positive binary.
3. If the sign bit is 1, the number is negative. Flip every bit, add 1, convert the result to decimal, and place a minus sign in front.

Example F: Decode 1111 1101 as 8-bit two's complement

Sign: The leftmost bit is 1, so the number is negative.

Flip: 1111 1101 → 0000 0010.

Add 1: 0000 0010 + 1 = 0000 0011.

Convert: 0000 0011 = 3.

Answer: 1111 1101 represents -3 in 8-bit two's complement.

6.6 The two's-complement range

Two's complement uses the bit pattern that one's complement spent on -0 to represent one additional negative number.

Formula: For n bits, two's complement ranges from $-2^{(n-1)}$ through $+(2^{(n-1)} - 1)$.

- 4 bits: -8 through +7.
- 8 bits: -128 through +127. The lecture explicitly contrasts this with one's complement, which is -127 through +127.

6.7 The 4-bit table from the lecture, expanded

4-bit pattern	Two's-complement decimal	Why
0000	0	Sign bit 0; ordinary binary 0.
0001	1	Sign bit 0; ordinary binary 1.
0010	2	Sign bit 0; ordinary binary 2.
0011	3	Sign bit 0; ordinary binary 3.
0100	4	Sign bit 0; ordinary binary 4.
0101	5	Sign bit 0; ordinary binary 5.
0110	6	Sign bit 0; ordinary binary 6.
0111	7	Largest positive 4-bit signed value.
1000	-8	Most negative 4-bit value.
1001	-7	Flip 0110, add 1 -> 0111 = 7.
1010	-6	Flip 0101, add 1 -> 0110 = 6.
1011	-5	Flip 0100, add 1 -> 0101 = 5.
1100	-4	Flip 0011, add 1 -> 0100 = 4.
1101	-3	Flip 0010, add 1 -> 0011 = 3.
1110	-2	Flip 0001, add 1 -> 0010 = 2.
1111	-1	Flip 0000, add 1 -> 0001 = 1.

7. Two's-Complement Addition: Every Lecture Example

Once the operands are already in two's-complement form, the hardware-style addition itself is ordinary binary addition. The special rule is simple: if a carry falls off the left edge, discard it. Do not wrap it around.

Two's-complement carry rule: Add the n-bit patterns normally. Keep only the lowest n bits. Any carry beyond the leftmost bit is discarded.

Example 8: Add -8 and +5 [Lecture p. 19]

Step 1: Build -8 in two's complement. $+8 = 0000\ 1000$. Flip $\rightarrow 1111\ 0111$. Add 1 $\rightarrow 1111\ 1000$.

Step 2: Write +5 normally: $0000\ 0101$.

Step 3: Add.

```
1111 1000 (-8)
+ 0000 0101 (+5)
-----
1111 1101
```

Step 4: Decode $1111\ 1101$ because the sign bit is 1. Flip $\rightarrow 0000\ 0010$. Add 1 $\rightarrow 0000\ 0011 = 3$.

Answer: $-8 + 5 = -3$, stored as $1111\ 1101$ in 8-bit two's complement.

Compare with one's complement: The decimal answer is still -3, but the stored bits are different. One's complement used $1111\ 1100$; two's complement uses $1111\ 1101$.

Example 9: Add -8 and -5 and discard the carry [Lecture p. 20]

Step 1: -8 in two's complement is $1111\ 1000$.

Step 2: Build -5. $+5 = 0000\ 0101$. Flip $\rightarrow 1111\ 1010$. Add 1 $\rightarrow 1111\ 1011$.

Step 3: Add the two bit patterns.

```
1111 1000 (-8)
+ 1111 1011 (-5)
-----
1 1111 0011 <- 9-bit raw sum
```

Step 4: Because this is two's complement, the leftmost carry is actually discarded. Keep only $1111\ 0011$.

Step 5: Decode $1111\ 0011$. Flip $\rightarrow 0000\ 1100$. Add 1 $\rightarrow 0000\ 1101 = 13$.

Answer: $-8 + -5 = -13$, stored as $1111\ 0011$ in 8-bit two's complement.

Do not do this: Do not add the discarded carry back into the result. That wraparound belongs to one's-complement addition only.

7.1 A useful shortcut for forming a negative two's-complement value

The safest method is always flip + add 1. Once you are comfortable, this equivalent shortcut can save time:

1. Starting from the rightmost bit, copy bits exactly until you copy the first 1.

2. After that first 1, flip every remaining bit to the left.

Example G: Shortcut: turn +13 = 0000 1101 into -13

+13: 0000 1101

From right: copy ...01 through the first 1; then flip all bits to its left

-13: 1111 0011

Recommendation: Use flip + add 1 on exams unless you are very comfortable. The shortcut is fast, but the standard method is easier to explain and check.

8. Padding and Sign Extension

The lecture states that unsigned numbers are padded with 0s, while signed numbers are padded with the sign bit. This matters when you increase the width of a number, such as changing 4 bits to 8 bits.

8.1 Unsigned or positive values

Pad with 0s on the left.

4-bit 5: 0101

8-bit 5: 0000 0101

8.2 Negative signed values

Repeat the sign bit on the left. A negative complement number has sign bit 1, so add 1s on the left.

Example H: Sign-extend -3 in one's complement

4-bit form: +3 = 0011, so -3 in 4-bit one's complement is 1100.

Extend to 8 bits: Repeat the sign bit 1 on the left: 1111 1100.

Check: Flip 1111 1100 -> 0000 0011 = 3, so the value stayed -3.

Example I: Sign-extend -3 in two's complement

4-bit form: +3 = 0011; flip -> 1100; add 1 -> 1101.

Extend to 8 bits: Repeat the sign bit 1: 1111 1101.

Check: Flip 1111 1101 -> 0000 0010; add 1 -> 0000 0011 = 3. The value stayed -3.

Common mistake: Padding a negative signed number with 0s on the left usually changes its value and can turn it positive. Use sign extension instead.

9. Overflow: When the True Answer Does Not Fit

The lecture notes that its +8 + +5 example has no overflow, but it does not fully teach overflow detection. This section expands that idea because overflow is a common question with signed binary arithmetic.

Overflow means the mathematically correct answer is outside the range that the fixed number of bits can represent.

- 8-bit one's complement range: -127 to +127.
- 8-bit two's complement range: -128 to +127.

9.1 Two's-complement overflow rule

Fast rule: When adding two signed two's-complement numbers, overflow can occur only when the two inputs have the same sign. If two positive inputs produce a negative-looking result, or two negative inputs produce a positive-looking result, overflow occurred.

Example 10: 8-bit two's complement: +100 + +50

Range check first: The true decimal result is 150. But 8-bit two's complement can only go up to +127, so overflow must occur.

Convert operands: 100 = 0110 0100 and 50 = 0011 0010.

```
0110 0100 (+100)
+ 0011 0010 (+50)
-----
1001 0110
```

What looks wrong: Both inputs have sign bit 0, meaning positive. The stored result begins with 1, which would be interpreted as negative. The sign flip reveals overflow.

Conclusion: 1001 0110 is not a valid representation of the intended +150 in 8-bit signed two's complement. The operation overflowed.

Carry is not the same as overflow: A final carry bit alone does not tell you whether signed overflow occurred. In two's complement, the carry-out is discarded; overflow is about whether the signed result fits the allowed range.

10. One's Complement vs. Two's Complement

Feature	One's complement	Two's complement
How to form -x	Write +x in n bits, then flip every bit.	Write +x in n bits, flip every bit, then add 1.
Positive values	Same ordinary binary form.	Same ordinary binary form.
Sign bit	0 = nonnegative; 1 = negative.	0 = nonnegative; 1 = negative.
Zero	Two encodings: +0 = all 0s, -0 = all 1s.	One encoding: all 0s.
8-bit range	-127 to +127.	-128 to +127.
Carry beyond left edge during addition	Wrap it around and add 1 to the low end.	Discard it.
Decode a negative	Flip bits; convert magnitude; add minus sign.	Flip bits; add 1; convert magnitude; add minus sign.
1111 1111 means	-0.	-1.

10.1 The exam decision tree

Step 0: Write the bit width at the top of your scratch work: 4-bit? 8-bit? 16-bit? Never skip this.

1. What representation did the question specify: unsigned, one's complement, or two's complement?
2. Are you ENCODING a decimal negative number into bits, DECODING bits into decimal, or ADDING bit patterns?
3. If encoding one's complement: write positive magnitude -> flip bits -> stop.

4. If encoding two's complement: write positive magnitude -> flip bits -> add 1.
5. If decoding one's complement and sign bit = 1: flip -> convert -> attach minus sign.
6. If decoding two's complement and sign bit = 1: flip -> add 1 -> convert -> attach minus sign.
7. If adding one's complement: normal binary addition -> if carry leaves left edge, wrap it around and add 1.
8. If adding two's complement: normal binary addition -> discard carry past left edge.
9. Finally, check the range for overflow if the question involves signed addition.

10.2 Translate common quiz wording

If the question says...	It is asking you to...
Find the one's complement of 0101 0011.	Flip every bit. No +1.
Represent -N in 8-bit one's complement.	Convert +N to 8 bits, then flip.
Represent -N in 8-bit two's complement.	Convert +N to 8 bits, flip, then add 1.
What decimal value does this signed one's-complement pattern represent?	Use sign bit; if negative, flip to recover magnitude.
What decimal value does this signed two's-complement pattern represent?	Use sign bit; if negative, flip + 1 to recover magnitude.
Add using one's complement.	Binary add, then apply end-around carry if needed.
Add using two's complement.	Binary add and discard any final carry-out.
Sign-extend to 8 bits.	Repeat the sign bit on the left for a signed number.

10.3 Common mistakes and how to prevent them

Mistake	Prevention
Forgetting the bit width	Always write the full fixed-width pattern before flipping.
Adding 1 in one's complement	Do not. One's complement is flip only.
Forgetting +1 in two's complement	Use the three-word memory phrase: write, flip, add one.
Wrapping the carry in two's complement	Do not. Wrap only for one's complement.
Discarding the carry in one's complement without adding it back	Use end-around carry: lower n bits + 1.
Reading a negative pattern as if it were unsigned	Check the representation and sign bit first.
Treating the left bit as a simple minus sign	Negative complement patterns must be decoded, not merely stripped of the first bit.
Confusing carry-out with overflow	Check signed range/sign behavior, not just the carry bit.
Padding a negative signed number with zeros	Sign-extend by repeating the sign bit 1.

11. Extra Practice With Fully Worked Solutions

These problems are not additional claims from the lecture; they are practice constructed from the exact procedures taught in the lecture. Each solution shows every step so you can imitate the process.

Example P1: Convert decimal 22 to 8-bit binary

Goal: Create the positive 8-bit starting form that complement problems require.

Break 22 into powers of 2: $22 = 16 + 4 + 2$.

Place value:	128	64	32	16	8	4	2	1
Bit:	0	0	0	1	0	1	1	0
Answer:	0001 0110							

Answer: $22 = 0001\ 0110$.

Example P2: Find the one's complement of 0010 1101

Method: Flip every bit. This question asks only for the complement of the pattern, not a decimal interpretation.

Original:	0010 1101
Flipped:	1101 0010

Answer: 1101 0010.

Example P3: Represent -18 in 8-bit one's complement

Step 1: Convert +18. Since $18 = 16 + 2$, $+18 = 0001\ 0010$.

Step 2: Flip all bits because the target is one's complement.

+18:	0001 0010
-18:	1110 1101

Answer: -18 = 1110 1101 in 8-bit one's complement.

Example P4: Decode 1110 1010 as 8-bit one's complement

Step 1: The sign bit is 1, so the number is negative.

Step 2: Flip all bits: 1110 1010 -> 0001 0101.

Step 3: $0001\ 0101 = 16 + 4 + 1 = 21$.

Answer: 1110 1010 represents -21 in 8-bit one's complement.

Example P5: Add +9 and +6 in 8-bit one's complement

Step 1: Both values are positive: $9 = 0000\ 1001$ and $6 = 0000\ 0110$.

```
0000 1001 (+9)
+ 0000 0110 (+6)
-----
0000 1111 (+15)
```

Step 2: There is no carry beyond the left edge, so no end-around step is needed.

Answer: +15 = 0000 1111.

Example P6: Add -7 and +3 in 8-bit one's complement

Build -7: $+7 = 0000\ 0111$. Flip -> 1111 1000.

Write +3: 0000 0011.

```
1111 1000 (-7)
+ 0000 0011 (+3)
-----
1111 1011
```

Decode: Sign bit 1 -> flip 1111 1011 to 0000 0100 = 4 -> answer is -4.

Answer: $-7 + 3 = -4$, stored as 1111 1011.

Example P7: Add -7 and -4 in 8-bit one's complement

Build -7: $+7 = 0000\ 0111$ -> flip -> 1111 1000.

Build -4: $+4 = 0000\ 0100$ -> flip -> 1111 1011.

Add: A carry will leave the left edge, so be ready for end-around carry.

```
1111 1000 (-7)
+ 1111 1011 (-4)
-----
1 1111 0011 <- raw 9-bit sum
 1111 0011 <- keep lower 8 bits
+ 0000 0001 <- wrap carry
-----
1111 0100
```

Decode: Flip 1111 0100 -> 0000 1011 = 11. Therefore the value is -11.

Answer: $-7 + -4 = -11$, stored as 1111 0100 in one's complement.

Example P8: Represent -18 in 8-bit two's complement

Step 1: +18 = 0001 0010.

Step 2: Flip -> 1110 1101.

Step 3: Add 1 -> 1110 1110.

```
1110 1101
+ 0000 0001
-----
1110 1110
```

Answer: -18 = 1110 1110 in 8-bit two's complement.

Example P9: Decode 1110 1010 as 8-bit two's complement

Step 1: The sign bit is 1, so the value is negative.

Step 2: Flip: 1110 1010 -> 0001 0101.

Step 3: Add 1: 0001 0101 + 1 = 0001 0110.

Step 4: 0001 0110 = 16 + 4 + 2 = 22.

Answer: 1110 1010 represents -22 in 8-bit two's complement.

Example P10: Add -7 and +3 in 8-bit two's complement

Build -7: +7 = 0000 0111; flip -> 1111 1000; add 1 -> 1111 1001.

Write +3: 0000 0011.

```
1111 1001  (-7)
+ 0000 0011  (+3)
-----
1111 1100
```

Decode: Flip 1111 1100 -> 0000 0011; add 1 -> 0000 0100 = 4 -> negative means -4.

Answer: -7 + 3 = -4, stored as 1111 1100 in two's complement.

Example P11: Add -7 and -4 in 8-bit two's complement

Build -7: 1111 1001.

Build -4: +4 = 0000 0100; flip -> 1111 1011; add 1 -> 1111 1100.

```
1111 1001  (-7)
+ 1111 1100  (-4)
-----
1 1111 0101  <- raw sum
1111 0101   <- discard left carry
```

Decode: Flip 1111 0101 -> 0000 1010; add 1 -> 0000 1011 = 11 -> result is -11.

Answer: -7 + -4 = -11, stored as 1111 0101 in two's complement.

Example P12: Recognize 8-bit two's-complement overflow: +100 + +50

Before adding: $100 + 50 = 150$, but the maximum 8-bit signed two's-complement value is +127. So the true answer cannot fit.

Binary sum: $0110\ 0100 + 0011\ 0010 = 1001\ 0110$.

Sign test: Both inputs were positive (sign bit 0), but the stored result has sign bit 1. That is the classic positive + positive -> negative-looking overflow pattern.

Answer: Overflow occurred. Do not report 1001 0110 as a valid +150 signed result.

12. Mini Self-Test

Try these without looking back. The short answers are immediately below, followed by what process you should have used.

- 1) What is the one's complement of 0101 1000?
- 2) Represent -12 in 8-bit one's complement.
- 3) Represent -12 in 8-bit two's complement.
- 4) Decode 1111 0110 as 8-bit one's complement.
- 5) Decode 1111 0110 as 8-bit two's complement.
- 6) In one's-complement addition, what do you do with a carry beyond the left edge?
- 7) In two's-complement addition, what do you do with a carry beyond the left edge?
- 8) What are the 8-bit signed ranges for one's complement and two's complement?

Self-Test Answers

- 1) 1010 0111. Flip every bit.
- 2) +12 = 0000 1100; flip -> 1111 0011.
- 3) +12 = 0000 1100; flip -> 1111 0011; add 1 -> 1111 0100.
- 4) Sign bit 1; flip 1111 0110 -> 0000 1001 = 9, so -9.
- 5) Sign bit 1; flip -> 0000 1001; add 1 -> 0000 1010 = 10, so -10.
- 6) Wrap it around and add 1 to the low end (end-around carry).
- 7) Discard it and keep only the allowed bit width.
- 8) One's complement: -127 to +127. Two's complement: -128 to +127.

13. Final One-Page Memory Sheet

ONE'S COMPLEMENT: Negative = write positive magnitude -> FLIP all bits -> STOP. Decode negative = FLIP -> convert -> minus sign. Addition = normal binary + END-AROUND CARRY. 8-bit range: -127 to +127. Two zeros: 0000 0000 and 1111 1111.

TWO'S COMPLEMENT: Negative = write positive magnitude -> FLIP all bits -> ADD 1. Decode negative = FLIP -> ADD 1 -> convert -> minus sign. Addition = normal binary + DISCARD final carry. 8-bit range: -128 to +127. One zero only.

Binary addition rules

```
0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10 -> write 0, carry 1
1 + 1 + 1 = 11 -> write 1, carry 1
```

8-bit place values

128	64	32	16	8	4	2	1
b7	b6	b5	b4	b3	b2	b1	b0

Five-second checklist before submitting an answer

- Did I use the exact required bit width?
- Did I identify one's vs. two's complement before touching the bits?
- If the number is negative, did I use the correct flip / +1 rule?
- If I added in one's complement, did I wrap any carry?
- If I added in two's complement, did I discard any carry?
- Did I decode the final negative pattern correctly?
- Does the decimal answer make sense, and is it inside the representable range?

Best way to study this: Cover the solution, solve one example completely on paper, then compare line by line. Do not move on until you can explain why each step happens. Once you can do $-8 + -5$ correctly in both systems and explain the different carry rule, you understand the core of the lecture.

Source Notes

Primary source used: the supplied CS 6306 slide deck titled 'One's and Two's Complement' (20 pages). The lecture's worked examples include bit flipping, -34 in one's complement, $+8 + +5$, $-8 + +5$, $-8 + -5$ with end-around carry, duplicated zero, two's-complement zero, two's-complement ranges, -34 in two's complement, and two's-complement signed addition.

Foundational binary explanations, the overflow section, and extra practice problems are instructional expansions written to make the lecture understandable to a beginner; they are not additional slide content.